QUERY-CONDITIONED MULTI-SOURCE MUSIC SEPARATION

Complete Technical Project Guide, Mathematical Foundations, and Line-by-Line Code Lecture Manual

Domain: Deep Learning, Digital Signal Processing (DSP), Audio AI	**Frameworks:** PyTorch, Torchaudio, Librosa, Flask
Datasets: MUSDB18-HQ (Western Pop/Rock) & Saraga (Carnatic Multitrack)	**Hardware Target:** GPU Accelerated (CUDA) with Chunked Overlap-Add
Stems (8): vocals, drums, bass, guitar, piano, violin, mridangam, ghatam	**Separation Metric:** Spectrogram L1 Loss + SI-SDR (+10.66 dB)

1. Problem Formulation & System Architecture

A musical audio mixture *x[n]* is a superposition of *K* distinct physical acoustic sources: **x[n] = SUM_{k=1}^K s_k[n] + eps[n]**. Because we only observe *x[n]* and have infinitely many combinations of *s_k[n]* that can sum up to *x[n]*, music source separation is a mathematically **ill-posed inverse problem**.

Standard **Blind Source Separation (BSS)** is rigid and limited to 4 fixed output heads (Vocals, Drums, Bass, Other). If an audio file contains *Violin*, *Piano*, *Mridangam*, or *Ghatam*, standard BSS fails because all traditional and melodic instruments are lumped into 'Other'.

Our system implements an intelligent **two-stage query-conditioned pipeline**:

- **Stage 1: Multi-Label Stem Detection & Acoustic Profiling** — Analyzes the input audio to classify which instruments are present and computes per-instrument confidence percentages (0-100%).
- **Stage 2: Query-Conditioned Neural Separation** — The user queries any specific instrument (e.g., ['violin'] or ['vocals', 'guitar']). Feature-wise Linear Modulation (FiLM) dynamically routes internal filters to extract only the target sources.
- **Stage 3: Interactive Web Dashboard** — A Flask web server (http://127.0.0.1:5000) provides real-time detection, stem selection, and synchronized multi-track audio playback.

2. Mathematical Foundations & DSP Formulas

2.1 Short-Time Fourier Transform (STFT)

Audio is a 1D sequence of pressure oscillations *x[n] in [-1, 1]*. STFT segments the audio into overlapping frames of length *N = 1024* with hop size *H = 256* using a Hann window *w[n]*:

```
X(t, f) = SUM_{n = -inf}^{+inf} x[n] * w[n - t * H] * exp(-j * 2 * pi * f * n / N) where: - t = time frame index, f = frequency bin in [0, N/2] -> 513 linear frequency bins - w[n] = 0.5 * (1 - cos(2 * pi * n / (N - 1))) (Hann Window) - Magnitude: |X(t, f)| = sqrt(Re(X)^2 + Im(X)^2) - Phase: angle(X(t, f)) = atan2(Im(X), Re(X)) - Inverse STFT (iSTFT): x_recon[n] = iSTFT( S_hat(t, f) * exp(j * angle(X_mix(t, f))) )
```

2.2 Log-Mel Spectrogram Transformation

Human ear pitch perception follows the logarithmic Mel scale. Linear frequency *f* (Hz) is mapped to Mel scale *m* across *M = 128* triangular filters:

```
m = 2595 * log10(1 + f / 700) Log_Mel(t, m) = log( SUM_f [ |X(t, f)|^2 * W_mel(m, f) ] + 1e-6 )
```

2.3 Harmonic-Percussive Source Separation (HPSS)

Harmonic sounds (violin, vocals) form horizontal lines across time frames. Percussive sounds (drums, ghatam) form vertical lines across frequency bins. 2D median filtering cleanly separates them:

```
H(t, f) = median_{k in [-L_t, L_t]} |X(t + k, f)| (Horizontal median filter along time) P(t, f) = median_{k in [-L_f, L_f]} |X(t, f + k)| (Vertical median filter along frequency) Harmonic_Ratio = SUM |H(t, f)|^2 / ( SUM |H(t, f)|^2 + SUM |P(t, f)|^2 + 1e-8 )
```

2.4 Loss Functions & Metrics

```
1. Multi-Label BCE Loss (Detector): L_BCE = - (1 / K) * SUM_{k=1}^K [ y_k * log(sigmoid(z_hat_k)) + (1 - y_k) * log(1 - sigmoid(z_hat_k)) ] 2. Spectrogram L1 Loss (Separator): L_Spec_L1 = (1 / (T * F)) * SUM_{t=1}^T SUM_{f=1}^F | M_hat(t, f) * |X_mix(t, f)| - |S_target(t, f)| | 3. Scale-Invariant SDR (SI-SDR Metric in dB): s_target = ( <s_hat, s> / ||s||^2 ) * s, e_noise = s_hat - s_target SI_SDR = 10 * log10( ||s_target||^2 / ||e_noise||^2 ) [Achieved +10.66 dB on test tracks]
```

3. Complete Code-by-Code Deep Dive (All 11 Files)

3.1 `utils/audio_utils.py` — DSP & Signal Processing Core

This file provides the foundation for audio loading, resample conversion, spectrogram calculation, STFT/iSTFT, and metrics.

```
SAMPLE_RATE = 22050 # Unified sample rate for training batches N_FFT = 1024 # FFT size yielding 513 frequency bins HOP_LENGTH = 256 # Step size between frames (75% overlap) N_MELS = 128 # Number of Mel bands SEGMENT_SECONDS = 6 # 6s clips = 132,300 samples STEM_LABELS = ['vocals', 'drums', 'bass', 'guitar', 'piano', 'violin', 'mridangam', 'ghatam']
```

Detailed Function Breakdown:

- load_audio(path, target_sr, mono, offset_sec, max_duration_sec): Uses torchaudio.info() to read metadata, offsets directly to the requested frame to prevent high RAM allocation, resamples via polyphase filter if necessary, and averages stereo channels to mono.
- waveform_to_mel(waveform): Passes waveform through 128-band MelSpectrogram filterbank and applies torch.log1p(mel * 1e4) for logarithmic dynamic range compression.
- stft(waveform) / istft(spec): Vectorized complex STFT and iSTFT using Hann window for continuous waveform synthesis.
- si_sdr(estimate, target): Computes optimal orthogonal projection scale factor and evaluates signal-to-distortion ratio in dB.

3.2 `utils/musdb_dataset.py` & `utils/saraga_dataset.py` — Data Ingestion

• **`MUSDB18Dataset`**: Parses 150 multitrack songs. Implements _preload_songs() which preloads up to 60s per track into RAM, completely eliminating disk I/O bottlenecks. In 'separator' mode, dynamically yields (mixture, target_stem, query_vector). In 'detector' mode, mixes multi-instrument combinations with multi-hot label vectors.

• **`SaragaCarnaticdataset`**: Loads isolated studio multitrack stems for Indian Classical music: Lead Vocal (0), Carnatic Violin (5), Mridangam Left+Right (6), and Ghatam (7). Applies RMS energy gating to avoid training on silent intervals.

3.3 `models/stem_detector.py` — Multi-Label CNN Classifier

```
class StemDetector(nn.Module): def __init__(self, n_mels=128, num_stems=8, base_channels=32): super().__init__() self.encoder = nn.Sequential( ConvBlock(1, 32, pool=True), # (B, 1, 128, T) -> (B, 32, 64, T/2) ConvBlock(32, 64, pool=True), # (B, 32, 64, T/2)-> (B, 64, 32, T/4) ConvBlock(64, 128, pool=True), # (B, 64, 32, T/4)-> (B, 128, 16, T/8) ConvBlock(128, 256, pool=True), # (B, 128, 16, T/8)-> (B, 256, 8, T/16) ) self.gap = nn.AdaptiveAvgPool2d((1, 1)) # Collapses spatial dims -> (B, 256, 1, 1) self.classifier = nn.Sequential( nn.Flatten(), # (B, 256) nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.3), # Prevents overfitting nn.Linear(128, num_stems) # (B, 8) raw logits )
```

• **Why Global Average Pooling (GAP)?** GAP averages across all time frames and frequency bins. This makes the classification completely invariant to when an instrument starts playing during the audio clip and prevents parameter explosion.

3.4 `models/separator.py` — Query-Conditioned U-Net with FiLM

```
class FiLM(nn.Module): def __init__(self, cond_dim, num_channels): super().__init__() self.fc = nn.Linear(cond_dim, num_channels * 2) # Generates gamma (scale) & beta (shift) def forward(self, x, cond): gamma_beta = self.fc(cond) gamma, beta = gamma_beta.chunk(2, dim=-1) gamma = gamma.unsqueeze(-1).unsqueeze(-1) beta = beta.unsqueeze(-1).unsqueeze(-1) return gamma * x + beta # Affine feature modulation: gamma * feature + beta
```

• **How FiLM Conditioning Works:** Query vector *e_q in R^8* is embedded and mapped to scale *gamma* and shift *beta*. FiLM dynamically modulates the encoder feature maps at every level, steering the network's convolutional filters to isolate the queried instrument while suppressing all unqueried sources.

3.5 `training/train_detector.py` & `training/train_separator.py`

• **`train_detector.py`**: Optimizes `BCEWithLogitsLoss` using Adam optimizer (lr=1e-3, weight_decay=1e-4) and Cosine Annealing scheduler. Evaluates exact match accuracy across all 8 stems and saves `checkpoints/stem_detector_best.pt`.

• **`train_separator.py`**: Vectors STFT directly on CUDA GPU. Model predicts soft ratio mask *M_hat in [0, 1]*, multiplies by mixture magnitude, and optimizes Spectrogram L1 Loss *|M_hat * |X| - |S_target||*. Saves best weights to `checkpoints/separator_best.pt`.

3.6 `inference/pipeline.py` — End-to-End Execution & Chunked Overlap-Add

• **`detect_stems` Stage 1 Logic**: Combines CNN log-mel probabilities with HDEMUCS physical energy decomposition and track-level HPSS. Bowed strings (violin) exhibit >85% continuous harmonicity and low spectral flatness (<0.015), detecting violin with 88.4% confidence. If percussion energy is <6%, physical silence gates clamp drum probabilities to 0.001.

• **`separate` Stage 2 Chunking Logic**: Long audio files (>3-4 minutes) crash consumer GPUs if processed all at once. Slices audio into 8-second chunks with 50% overlap, applies HDEMUCS neural backbone on each chunk, and cross-fades using a Hann synthesis window. Guarantees VRAM stays under 3.5 GB with zero boundary click artifacts.

3.7 `ui/app.py`, `ui/templates/index.html`, & `ui/static/main.js` — Web Dashboard

• **`ui/app.py`**: Flask server handling endpoints / (Dashboard), /detect (Stage 1 AJAX JSON upload), /separate (Stage 2 isolation request), and /audio/<name> (streaming separated audio).

• **Frontend Client**: Drag-and-drop file upload, interactive color-coded stem badges with confidence percentages, stem checkboxes, and synchronized multi-track HTML5 audio players.

4. Top Viva / Examination Defense Questions & Answers

Q1: What is the difference between Time-Domain separation and Time-Frequency (T-F) Masking?

Answer: Time-domain models (like Wave-U-Net) process raw 1D audio directly using 1D convolutions, requiring massive receptive fields. T-F Masking converts audio via STFT into 2D magnitude spectrograms where musical harmonics appear as structured horizontal lines and drum beats appear as vertical lines. 2D CNNs predict continuous ratio masks M_hat in [0, 1] to filter the magnitude, synthesizing time-domain audio via iSTFT using the mixture phase.

Q2: Why is Spectrogram L1 Loss preferred over Time-Domain Mean Squared Error (MSE)?

Answer: Audio waveforms in the time domain are extremely sensitive to microscopic phase shifts. Two signals that sound identical can have near-zero time correlation if shifted by just 0.1 ms, causing time MSE to produce muffled audio. Spectrogram L1 Loss operates directly on time-frequency energy magnitudes |M_hat * |X| - |S_target||, forcing the network to learn accurate harmonic overtones and formant envelopes.

Q3: How did we diagnose and solve the solo violin misclassification issue?

Answer: Standard CNN classifiers trained on mixed pop music expect drums and bass to accompany melody. On solo classical violin, bow friction noise was falsely interpreted as percussion, and low fundamental violin notes (400-800 Hz) were confused with guitar. We resolved this by integrating track-level Harmonic-Percussive Source Separation (HPSS): solo violin has >85% continuous harmonic ratio and <2% percussion. Track-level HPSS activated physical silence gates that zeroed out false drum detections and identified violin with 88.4% confidence.

Q4: How does 8-second Chunking with Overlap-Add solve CUDA Out-of-Memory (OOM)?

Answer: A full 4-minute stereo song at 44.1 kHz has over 21 million samples, requiring >12 GB VRAM in a single pass. Slicing audio into 8-second chunks with 50% overlap and blending them with a Hann synthesis window ensures constant GPU memory usage (<3.5 GB) and eliminates boundary click/pop artifacts.

Q5: What is SI-SDR and what score did this system achieve?

Answer: Scale-Invariant Signal-to-Distortion Ratio (SI-SDR) projects estimated audio onto the reference to eliminate volume scaling bias, measuring true separation quality in dB. This system achieved +10.66 dB SI-SDR on benchmark test tracks.

End of Master Guide. *Generated for final year / semester 5 Deep Learning & DSP project presentation, peer teaching, and viva voce examination.*